1주차 풀이 읽기자료

해설 사용 순서

먼저 학생 실습에서 문제를 독립적으로 풉니다. 이 자료는 힌트와 완전 해설을 포함합니다. 필요한 단계까지만 읽고 다시 풀이합니다.

표시된 예시는 강의에서 만든 예시입니다. 원문과 전체 제약은 각 공식 링크에서 확인합니다.

학생 실습

힌트·해설

교사용 진행안

공통 참고표

코드 전체 내려받기

1 · 단계별 힌트

LeetCode 1: Two Sum · Easy

신규 연습

H1. 현재 값과 짝이 되는 값은 무엇인가요?

H2. 같은 위치를 두 번 사용하지 않으려면 언제 기록하나요?

H3. 이전 값→인덱스를 저장하고 target-value를 조회한 뒤 현재 값을 기록합니다.

1 · 정당성과 비용

반복 시작 때 사전에는 앞서 본 위치만 있습니다. 답의 두 번째 위치에 도착하면 첫 위치가 기록되어 있으므로 답을 찾습니다. 모든 쌍을 검사하는 $O(n^2)$ 방법에서 이전 정보를 재사용한 것입니다.

n=원소 수. 평균 시간 O(n), 보조 공간 O(n). 해시 충돌의 최악에는 시간 $O(n^2)$.

1 · 반례와 검증

[3,3], target=6에서 현재 값을 먼저 저장하면 같은 인덱스를 두 번 반환할 수 있습니다.

```
nums=[4,8,3], target=7 → [0,2]
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

1 · 구현 1/1

H4. 아래 코드를 이어 붙이면 실행 파일의 완전한 구현과 같습니다.

```
def two_sum(nums, target):
    seen = {}
    for index, value in enumerate(nums):
        if target - value in seen:
            return [seen[target - value], index]
        seen[value] = index
    return []  # Defensive extension: official inputs have one answer.
```

219 · 단계별 힌트

LeetCode 219: Contains Duplicate II · Easy

신규 연습

H1. 같은 값의 어떤 위치를 남길까요?

H2. 가장 가까운 과거 위치보다 오래된 위치가 더 유리할까요?

H3. 값마다 최근 인덱스를 남기고 거리를 비교한 뒤 갱신합니다.

219 · 정당성과 비용

최근 위치는 모든 이전 위치 중 현재 위치와 가장 가깝습니다. 이 거리도 k보다 크면 다른 과거 위치도 실패합니다. 갱신으로 다음 반복에도 같은 성질이 유지됩니다.

n=길이. 평균 O(n) 시간, O(n) 보조 공간. 해시 최악 $O(n^2)$.

219 · 반례와 검증

최초 위치만 저장하면 [4,7,4,4], k=1에서 마지막 두 위치를 놓칩니다. k=0에서는 서로 다른 위치의 답이 없습니다.

```
nums=[4,7,4,4], k=1 → True
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

219 · 구현 1/1

H4. 아래 코드를 이어 붙이면 실행 파일의 완전한 구현과 같습니다.

```
def nearby_duplicate(nums, k):
    latest = {}
    for index, value in enumerate(nums):
        if value in latest and index - latest[value] <= k:
            return True
        latest[value] = index
    return False
```

167 · 단계별 힌트

LeetCode 167: Two Sum II - Input Array Is Sorted · Medium

신규 연습

H1. 양 끝 합이 작으면 어느 후보들이 모두 작나요?

H2. 정렬 조건을 지우면 이 제거가 가능한가요?

H3. left<right인 동안 합이 작으면 left, 크면 right를 안쪽으로 이동합니다.

167 · 정당성과 비용

합이 작으면 현재 left와 right 이하의 어떤 값도 답을 만들지 못합니다. 합이 크면 현재 right와 left 이상의 어떤 값도 실패합니다. 제거된 쌍에는 답이 없으므로 남은 범위가 답을 보존합니다.

n=길이. 각 포인터 이동 합 O(n), 시간 O(n), 보조 공간 O(1).

167 · 반례와 검증

정렬 전제 밖 [3,1,4], target=5에는 같은 이동 규칙을 적용할 수 없습니다. left==right는 같은 원소 재사용입니다.

```
numbers=[1,4,6,9], target=10 → [1,4]
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

167 · 구현 1/1

H4. 아래 코드를 이어 붙이면 실행 파일의 완전한 구현과 같습니다.

```
def two_sum_sorted(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

15 · 단계별 힌트

LeetCode 15: 3Sum · Medium

신규 연습

H1. 한 값을 고정하면 남는 문제는 무엇인가요?

H2. 같은 값을 다시 고정하면 새 조합이 생기나요?

H3. 복사 정렬 후 첫 값을 고정하고 뒤쪽에서 두 포인터를 사용합니다. 고정 값과 발견한 쌍의 중복을 건너뜁니다.

15 · 정당성과 비용

각 정렬된 세 값은 가장 작은 값의 차례에서 검사됩니다. 두 포인터 제거는 정렬로 정당화됩니다. 같은 값을 건너뛰는 것은 위치가 다른 동일한 값 조합만 제거합니다.

n=길이, z=출력 조합 수. $O(n^2)$ 시간, 복사·정렬 보조 O(n), 출력 O(z).

15 · 반례와 검증

[0,0,0,0]은 한 조합만 반환합니다. 값이 같다는 이유로 서로 다른 위치를 금지하면 이 답을 잃습니다.

`nums=[-2,0,1,1,2] → [[-2,0,2],[-2,1,1]]`

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

15 · 구현 1/2

H4. 아래 코드를 이어 붙이면 실행 파일의 완전한 구현과 같습니다.

```
def three_sum(nums):
    values = sorted(nums)
    result = []
    for index in range(len(values) - 2):
        if index and values[index] == values[index - 1]:
            continue
        left, right = index + 1, len(values) - 1
        while left < right:
            total = values[index] + values[left] + values[right]
            if total < 0:
                left += 1
```

15 · 구현 2/2

H4. 아래 코드를 이어 붙이면 실행 파일의 완전한 구현과 같습니다.

```
            elif total > 0:
                right -= 1
            else:
                result.append([values[index], values[left], values[right]])
                left += 1
                right -= 1
                while left < right and values[left] == values[left - 1]:
                    left += 1
                while left < right and values[right] == values[right + 1]:
                    right -= 1
    return result
```

실행과 참고자료

```
cd code
python demo_01.py
python demo_02.py
python verify.py
```

완성 코드는 `solutions.py`, 독립 실습은 `student_week_01.py`에 있습니다. LeetCode에서는 공식 메서드 서명에 맞춰 함수를 연결합니다.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).